

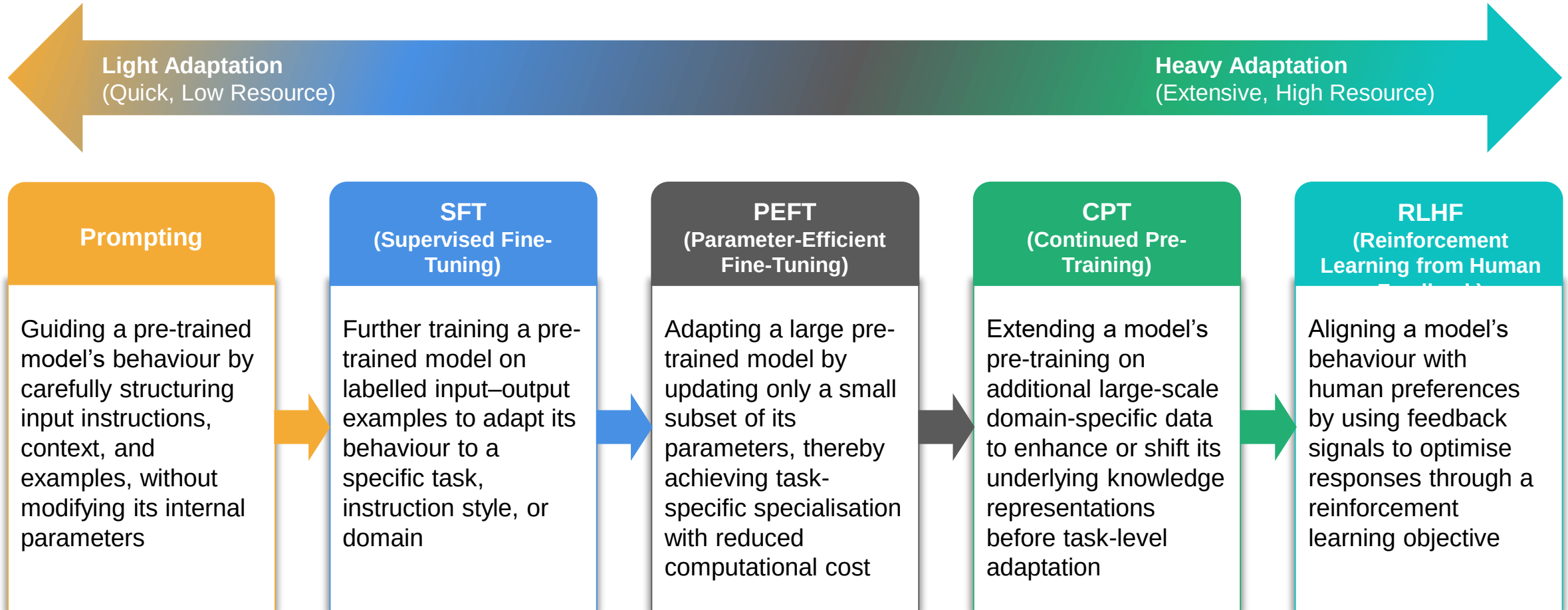
MODULE 3: Model Adaptation and Fine-Tuning, and Evaluation

Session 1: Model Adaptation Basics

Adaptation Spectrum

ADAPTATION SPECTRUM

Different adaptation methods sit on a single spectrum from “no parameter changes” to “heavy retraining.” It helps to think in terms of **what** changes: **behaviour** (how the model acts and follows instructions) vs **knowledge** (what facts, domains, or skills it has)



ADAPTATION SPECTRUM

	Prompting	SFT (Supervised Fine-Tuning)	PEFT (Parameter- Efficient Fine- Tuning)	CPT (Continued Pre- Training)	RLHF (Reinforcement Learning from Human Feedback)
What Changes	Behaviour (via context)	Behaviour + some knowledge	Behaviour + task- specific knowledge	Knowledge (domain/facts)	Behaviour (preferences, safety)
Parameters Updated	None	All (full FT) or many (PEFT)	Small subset	All or many	All or many (often after SFT)
Typical Use	Quick tryouts, one-off tasks	Instruction following, style, format	Domain/task adaptation with limited data/compute	Domain expertise, new facts	Alignment, preference, safety
Compute Cost	Very low (inference only)	High	Moderate	Very high	Very high

BEHAVIOUR VS KNOWLEDGE ADAPTATION

Behaviour (How the model responds)

- Tone
- Output Format
- Instruction-following
- Helpfulness preferences
- Safety and refusal patterns



Mainly Targeted By

Prompting

SFT
(Supervised
Fine-Tuning)

RLHF
(Reinforcement
Learning from
Human Feedback)

Knowledge (What the model knows)

- Factual knowledge
- Domain expertise
- Language coverage
- Reasoning patterns



Mainly Targeted By

CPT
(Continued
Pre-Training)

SFT
(Supervised
Fine-Tuning)

PEFT
(Parameter-Efficient
Fine-Tuning)

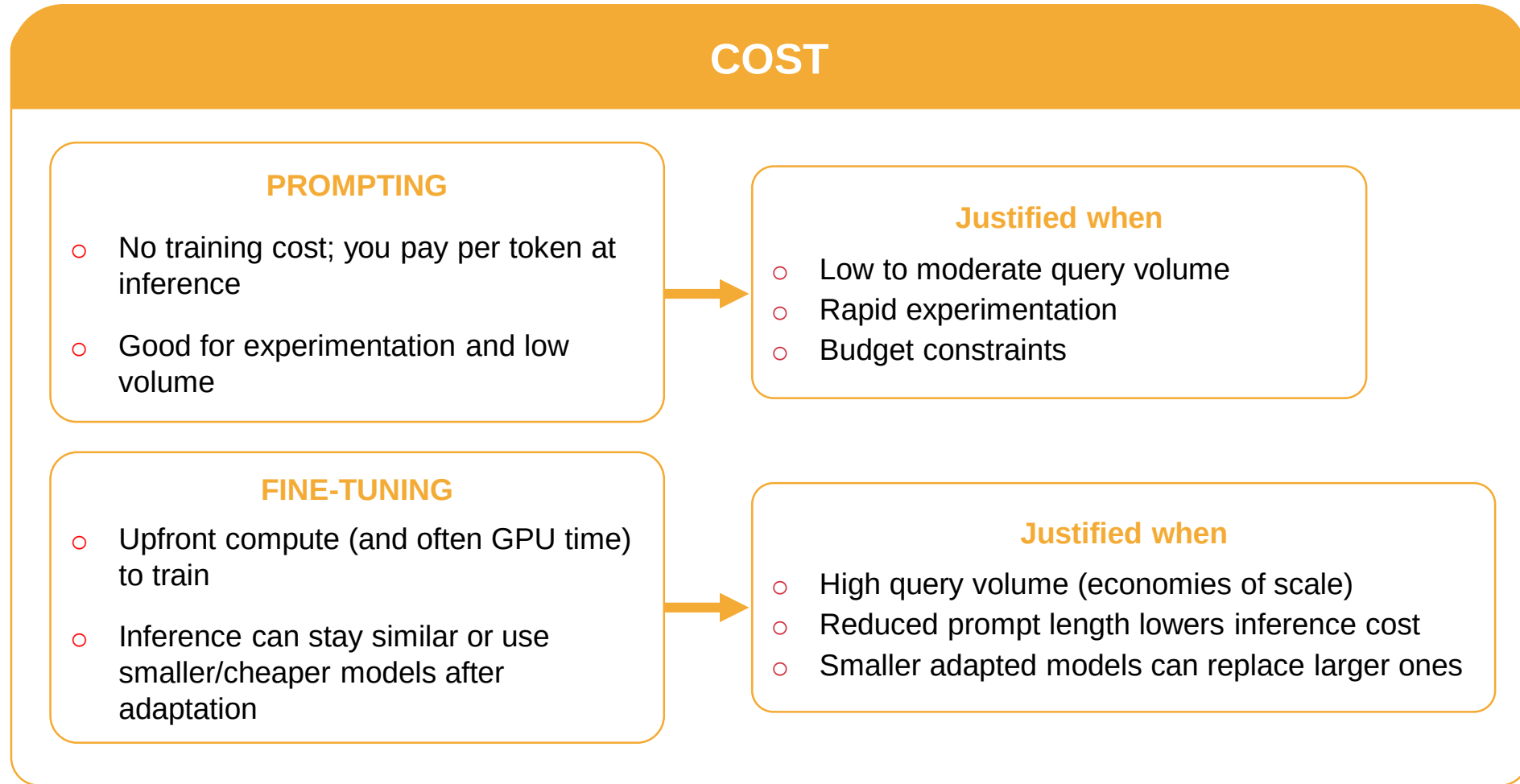
Prompting surfaces existing knowledge via context, but does not modify internal parameters

Prompting and **RLHF** primarily shape behaviour; **CPT** primarily modifies knowledge; **SFT** and **PEFT** can influence both

Decision Making for Fine-Tuning

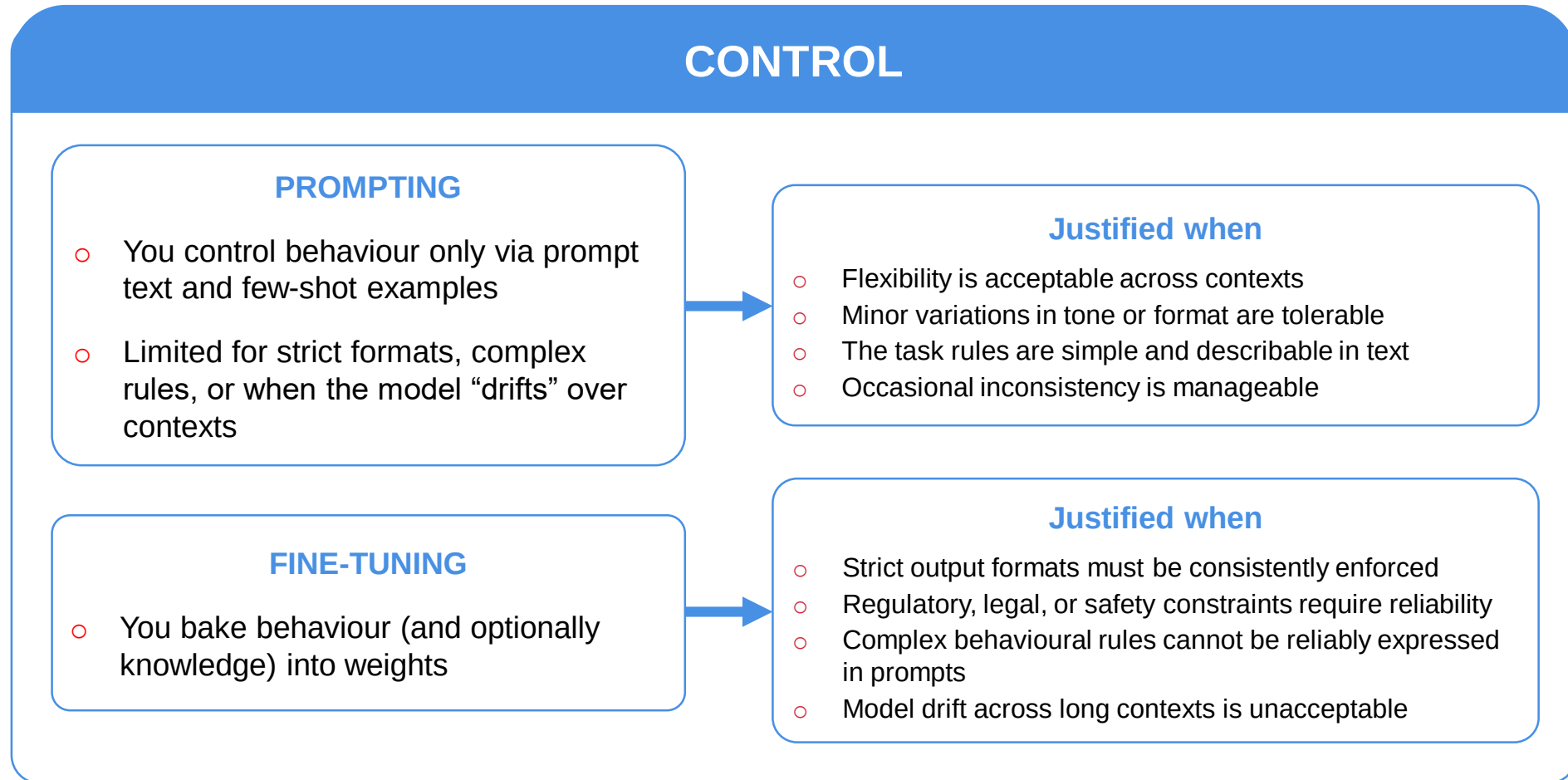
WHEN IS FINE-TUNING JUSTIFIED?

As fine-tuning is an expensive process, use three criteria to decide when to fine-tune: **cost**, **control**, and **data**



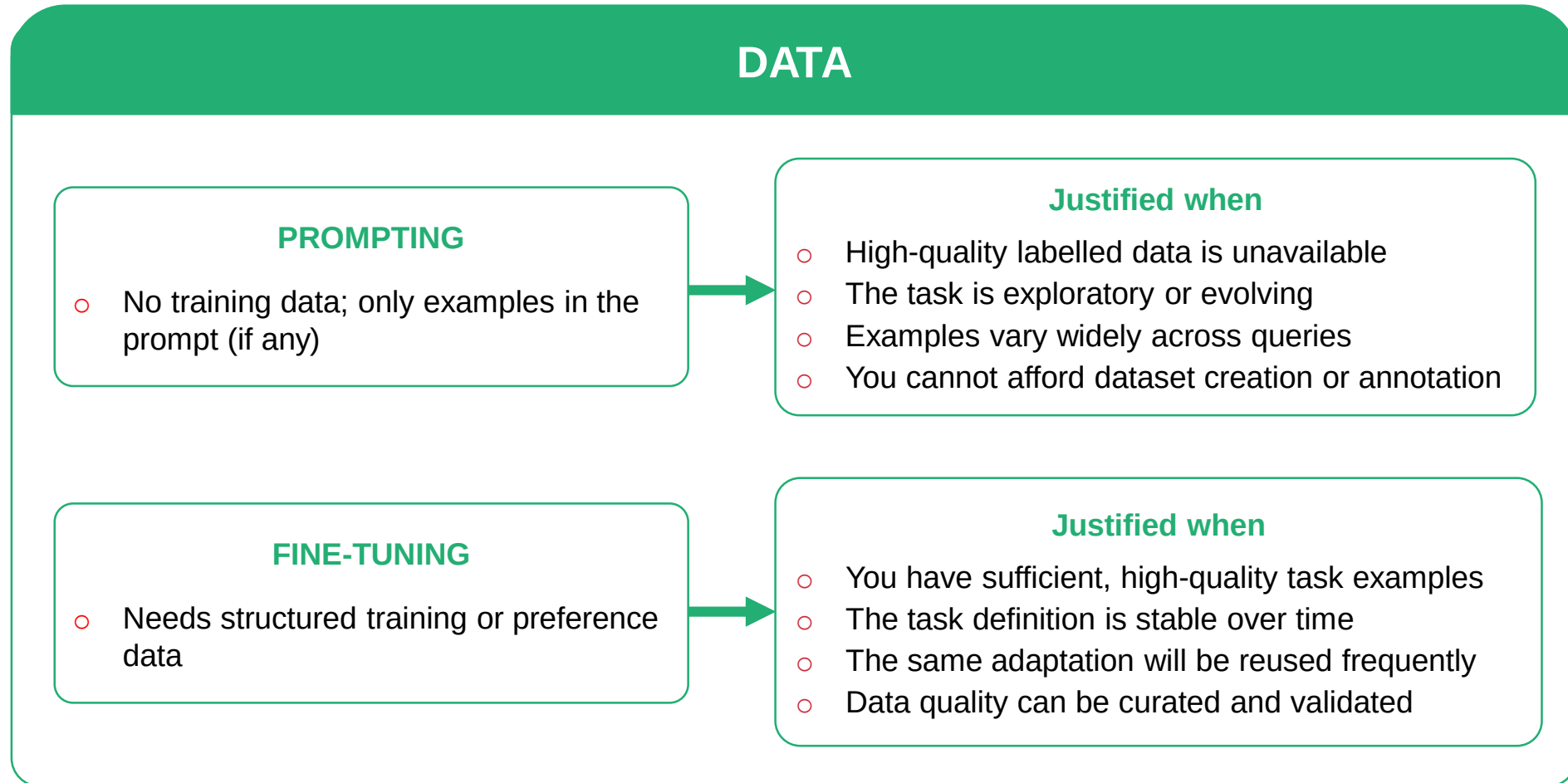
Fine-tuning is a capital expense; prompting is an operational expense

WHEN IS FINE-TUNING JUSTIFIED?



Fine-tuning increases reliability across contexts, not just within a single prompt

WHEN IS FINE-TUNING JUSTIFIED?



Poor-quality data will degrade model performance more severely than poor prompts

WHEN IS FINE-TUNING JUSTIFIED?

RULE OF THUMB

- Try prompting first
- If control or consistency is insufficient, evaluate PEFT
- Move to full fine-tuning or CPT only if knowledge gaps are clear

Parameter-Efficient Fine-Tuning

OVERHEAD OF FINE-TUNING

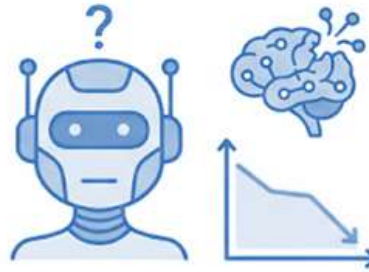
Fine-tuning is not always necessary and has several overheads, such as **compute**, **risk**, and **storage**

COMPUTE



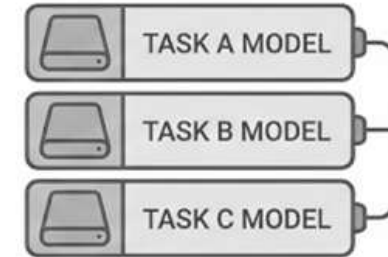
- Updating all parameters (e.g., 7B-70B+) requires large GPU memory and long training
- Most organisations can't afford it

RISK



- Full fine-tuning can cause catastrophic forgetting of general skills: the model overfits to your data and loses breadth
- It's also easy to degrade safety or instruction following

STORAGE

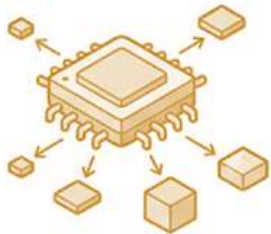


- One full copy of the model per task or variant is expensive

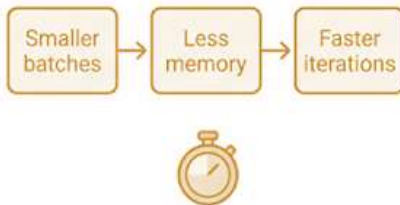
PARAMETER-EFFICIENT FINE-TUNING (PEFT)

- Parameter-Efficient Fine-Tuning (PEFT) techniques adapt LLMs to new tasks by training only a tiny subset of parameters (often $<1\%$) while freezing the base model
- This drastically reduces computational costs, memory requirements, and training time, while avoiding catastrophic forgetting

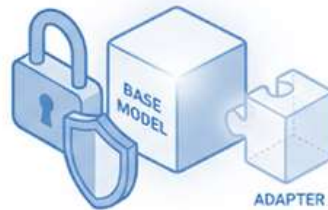
LESS COMPUTE



Fewer parameters to optimise, resulting in smaller gradients



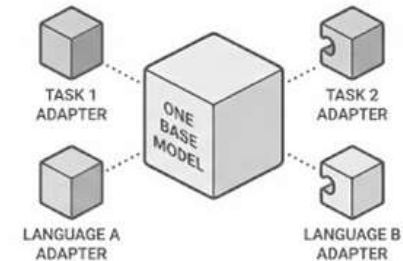
LESS RISK



The base model stays intact; you add a small “adapter” that specialises the model



REUSE



One base model, many small adapter weights (e.g. one per task or language), which are cheap to store and swap



PEFT: APPROACHES

Instead of updating all billions of parameters, PEFT adds small, trainable, adaptive layers or modifies existing weights to learn task-specific information

- **The following are the approaches towards PEFT**

- **LoRA (Low-Rank Adaptation):** Introduces small trainable low-rank matrices into transformer layers, significantly reducing the number of trainable parameters
- **QLoRA:** A highly efficient variant that quantises the base model to 4-bit to further minimise VRAM usage
- **Prefix Tuning/Prompt Tuning:** Adds learnable vectors to input sequences to guide model behaviour
- **Adapter Layers:** Inserts small, fully connected networks between existing transformer layers

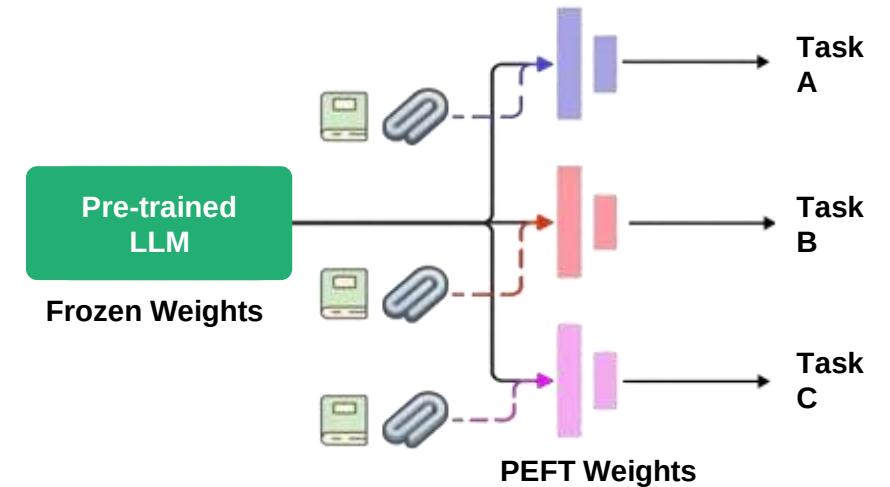
PEFT: APPROACHES

Method	Modifies	Typical Use
LoRA	Weight matrices	General task adaptation
QLoRA	Quantised base + LoRA	Low VRAM training
Prefix/Prompt Tuning	Input embeddings	Lightweight behavioural shift
Adapters	Inserted layers	Modular multi-task systems

Rule of thumb: PEFT is the default choice when you want fine-tuning but have limited compute or data, or want to keep the base model stable

PEFT: BENEFITS

- **Efficiency:** Drastically reduces GPU VRAM requirements and storage
- **Performance:** Achieves results comparable to full fine-tuning
- **Versatility:** Prevents catastrophic forgetting, ensuring the base model retains its original capabilities
- **Scalability:** One base model supports many task adapters



Rule of thumb: PEFT is ideal for adapting pre-trained models to domain-specific applications (e.g., medical, legal) with limited hardware or data

Low Rank Adaptation (LoRA)

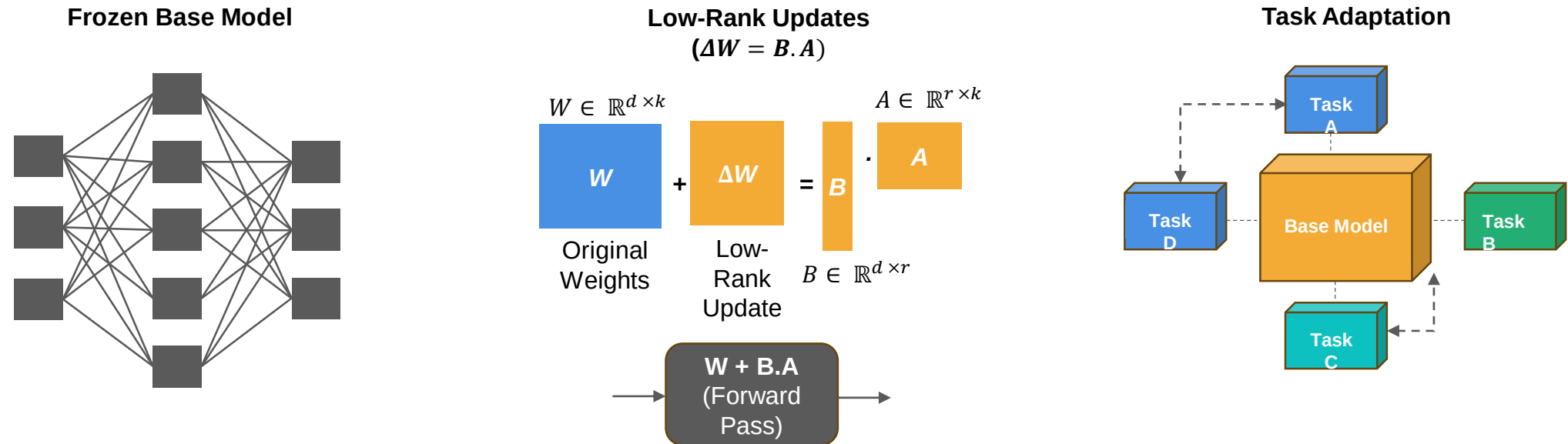
LOW RANK ADAPTATION (LoRA)

LoRA is one of the most widely used PEFT methods

LoRA assumes that task-specific updates lie in a low-dimensional subspace of the full parameter space.

The intuition is in three parts

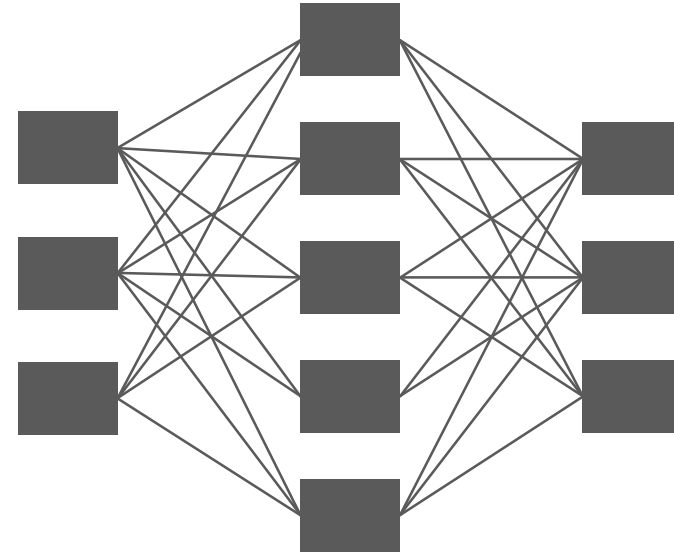
- Frozen base
- Low-rank updates
- Task adaptation



PART I: FROZEN BASE MODEL

- The pre-trained model weights **stay fixed**
- No gradients flow through them; they remain the shared “backbone” that already encodes language and reasoning
- There is no risk of overwriting that knowledge with a small task-specific dataset
- This preserves general language modelling capability and prevents catastrophic forgetting

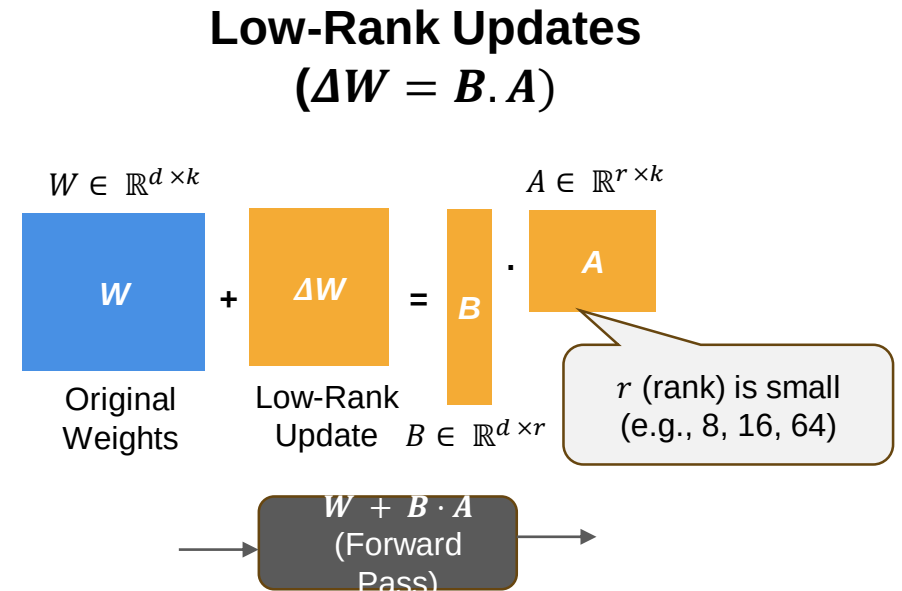
Frozen Base Model



- Pre-trained weights (W) stay fixed
- No gradients
- Shared backbone encoding language and reasoning
- No catastrophic forgetting

PART II: LOW RANK UPDATES

- Instead of updating the original weight matrix W , LoRA adds a low-rank update ΔW that is factorised as two small matrices B and A
 - $\Delta W = B \cdot A$
- Forward pass uses $W' = W + B \cdot A$ (base + adapter)
- Only A and B are trained; W is frozen

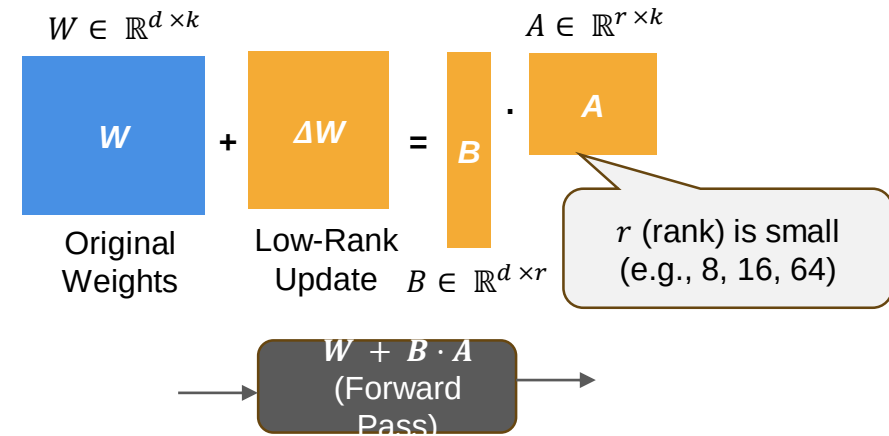


PART II: LOW RANK UPDATES

Why low-rank?

- Research suggests that when adapting to a new task, the *effective* change in the weight space often has low rank, i.e. a few directions in parameter space matter
- So we only need to learn those directions (via A and B) instead of every entry of W . That gives huge parameter savings and often matches full fine-tuning quality for many tasks

Low-Rank Updates ($\Delta W = B \cdot A$)



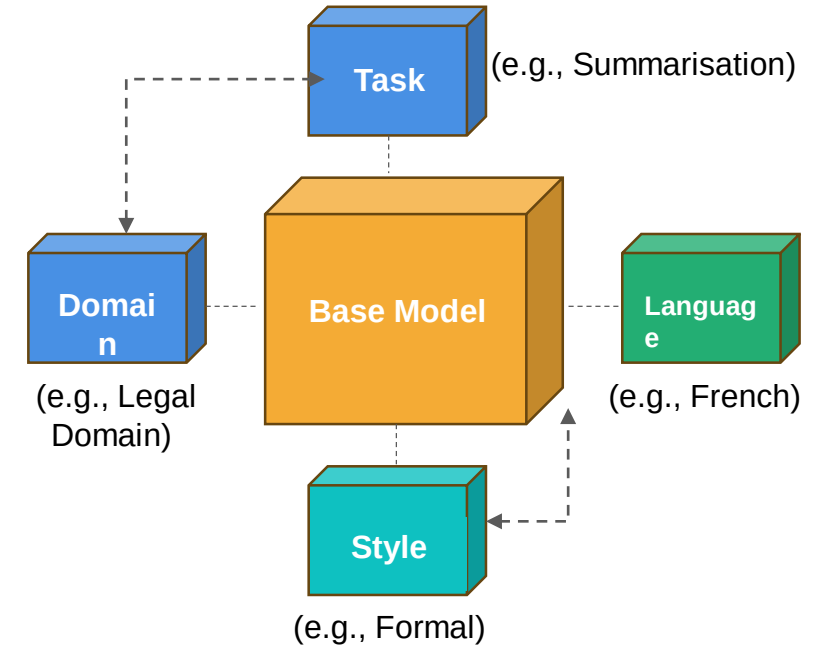
- Huge parameter savings
 - **Full fine-tuning:** learns $d \times k$ parameters
 - **LoRA:** learns only $2 \times d \times r$ parameters
 - Since $r \ll k$, we have $2dr \ll dk$
- Learns effective task-specific directions

PART III: TASK ADAPTATION

LoRA learns **task-specific** (or domain-specific) directions

- At inference, we can
 - Use the base model alone (no LoRA)
 - Attach one or more LoRA adapters for a given task, language, or style
- Multiple LoRA adapters can be composed or swapped dynamically at inference time

Task Adaptation



- One base model + many small LoRA files
- Flexible, efficient adaptation at inference without retraining the whole model

LOW RANK ADAPTATION

LoRA = frozen base + low-rank updates + task-specific adapters

- Fewer parameters
- Less compute
- Less forgetting
- Often matches full fine-tuning performance while training <1% of parameters